models.factory

Arquitetura, fundamentos e governança da Fábrica Preditiva

Equipe models.factory

20/10/2026

Índice

1 Fundamentos científicos e arquitetura **4**
1.1 Origem e escopo 4
1.2 Aprendizado estatístico e regularização 4
1.3 Árvores e Gradient Boosting 5
1.3.1 XGBoost 5
1.3.2 LightGBM 6
1.4 Linguagem de orquestração e motores computacionais 6
1.5 Expansibilidade e saídas 6

2 Governança e qualidade de dados **8**
2.1 Dimensões e responsabilidades 8
2.2 Data Quality Score 8
2.3 pointblank e gates 9

3 Modelagem de dados e álgebra relacional **10**
3.1 Transformações da Feature Store 10
3.2 Covariância e correlação 10
3.3 Entidades do projeto 11
3.4 Modelo Entidade-Relacionamento 11
3.5 Dicionário de tabelas e campos 12
3.5.1 ML_METHOD_LIBRARY 12
3.5.2 ML_FEATURE_LINEAGE 12
3.5.3 ML_FEATURE_STORE 12
3.5.4 ML_MODEL_REGISTRY 12
3.5.5 ML_MODEL_ARTIFACT 12
3.5.6 ML_MODEL_TARGET 12
3.5.7 ML_PREDICTION_RESULT 12

4 Infraestrutura de alto desempenho **13**
4.1 Hardware, sistema operacional e memória 13
4.2 Intel oneAPI e concorrência 13
4.3 Compilação e controle numérico 14
4.4 Processamento na origem e Oracle In-Memory 15

5 Governança bancária e auditoria **16**
5.1 Aplicabilidade regulatória 16
5.2 BCBS 239 e dados de risco 16
5.3 Risco de modelos e evidências 17

6 Padrões de metadados, linhagem e contratos **18**
6.1 OpenLineage, Dublin Core e JSON Schema 18
6.2 OpenLineage e decomposição da SQL 18
6.3 Vocabulário Dublin Core 19
6.4 JSON Schema e constraints 19
6.4.1 Exemplo hipotético para ML_FEATURE_LINEAGE 20

6.5 Interoperabilidade financeira e FAIR 22

7 Engenharia de software e reprodutibilidade 23
7.1 Qualidade de produto e ciclo de vida 23
7.2 MLOps, DataOps e DAGs 23
7.3 Reprodutibilidade, integridade e imutabilidade 24

8 Persistência e precisão no Oracle Database 19c EE 25
8.1 Responsabilidades e organização física 25
8.2 Política de precisão decimal e não uso de FLOAT 25
8.3 SecureFiles e políticas de armazenamento 26
8.4 Formatos e catálogo polimórfico 27
8.5 Explicabilidade e quantização 27
8.6 Consulta de arquivos externos 28

9 Preservação dupla do modelo final 29
9.1 Modelo nativo do XGBoost 29
9.2 ONNX como representação de intercâmbio 29
9.3 Contrato de preservação e equivalência 30

10 Evolução tecnológica e marcadores de portabilidade 31
10.1 Continuidade dos padrões 31
10.2 JSON Schema declarativo 31
10.3 ONNX e XGBoost no ecossistema Oracle 32
10.4 Lakehouse e vetores 32

11 Matriz de aderência e evolução arquitetural 33
11.1 Requisitos, decisões e evidências 33
11.2 Roadmap e estados 34

12 Modelagem 35

13 Predictive Factory Pipeline 36

14 Referência de funções 37

Referências 38

A Fábrica Preditiva organiza o ciclo de vida de modelos supervisionados para a gestão da carteira de clientes bancários. A arquitetura integra dados, métodos, execuções e artefatos verificáveis, com processamento analítico no Oracle Database 19c Enterprise Edition e orquestração em R.

A preservação dupla do modelo final, em formato nativo do XGBoost e ONNX, tem como objetivo ampliar a portabilidade. A execução de modelos no Oracle não integra o objetivo desta arquitetura. O suporte documentado em versões posteriores constitui evidência adicional da adoção dos padrões escolhidos.

1 Fundamentos científicos e arquitetura

1.1 Origem e escopo

A origem metodológica vincula-se às iniciativas de pesquisa da UFRGS sobre gestão analítica da carteira de clientes. O estudo de evasão publicado em *Financial Innovation* combina engenharia de variáveis, tratamento do desbalanceamento e modelos supervisionados. A pesquisa sobre Open Banking investiga a propensão ao compartilhamento de dados e sua interpretação. São trabalhos distintos: o DOI `10.1186/s40854-023-00558-3` identifica o artigo sobre evasão de 2024; o trabalho sobre Open Banking está documentado no preprint `10.48550/arXiv.2507.01987` (Brito et al., 2024; Brito et al., 2025).

Aplicação	Evento de interesse	Uso analítico
Evasão de clientes, ou churn	Perda do relacionamento segundo regra de negócio	Priorização de ações de retenção
Inflow Consent	Consentimento para entrada de dados de outras instituições	Antecipação da adesão ao compartilhamento
Outflow Consent	Consentimento para saída de dados para outras instituições	Gestão do relacionamento em ambiente competitivo

A definição operacional dos eventos, as janelas temporais e os experimentos serão desenvolvidos no capítulo **Modelagem**. A implementação dos componentes será documentada em **Predictive Factory Pipeline**.

1.2 Aprendizado estatístico e regularização

Considere observações $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{n}$, função preditiva f, perda ℓ e classe de funções $\mathcal{F}$. A aprendizagem procura reduzir o risco esperado $R(f) = \mathbb{E}[\ell(Y, f(X))]$, estimado pelo risco empírico. A regularização controla a complexidade da solução (Hastie; Tibshirani; Friedman, 2009; Vapnik, 1998):

$$\hat{f} = \arg\min_{f \in \mathcal{F}} \left\{ \frac{1}{n} \sum_{i=1}^{n} \ell(y_i, f(x_i)) + \lambda \Omega(f) \right\}.$$

O desempenho relevante é o observado em dados compatíveis com o uso futuro. Separação temporal inadequada, transformação ajustada com dados de teste e uso de informação posterior ao evento comprometem a avaliação.

Para regressão linear, uma parametrização do Elastic Net é (Zou; Hastie, 2005):

$$\hat{\beta} = \arg\min_{\beta} \left\{ \frac{1}{2n}\|y - X\beta\|_2^2 + \lambda \left[\alpha\|\beta\|_1 + \frac{1-\alpha}{2}\|\beta\|_2^2 \right] \right\}, \qquad \lambda \geq 0, \quad 0 \leq \alpha \leq 1.$$

A penalização L_1 favorece coeficientes nulos. A penalização L_2 estabiliza coeficientes correlacionados. A combinação atende cenários com muitas variáveis e multicolinearidade. Intercepto, padronização e convenção de escala da perda devem constar da configuração do método.

1.3 Árvores e Gradient Boosting

1.3.1 XGBoost

GBDT constrói um preditor aditivo de árvores. No XGBoost, cada iteração acrescenta uma árvore f_t, com penalização de complexidade (Chen; Guestrin, 2016):

$$F_t(x) = F_{t-1}(x) + \eta f_t(x), \qquad \mathcal{L}^{(t)} = \sum_{i=1}^{n} \ell(y_i, F_{t-1}(x_i) + f_t(x_i)) + \Omega(f_t).$$

A expressão da função objetivo apresenta a árvore antes da aplicação da taxa de aprendizagem η. Para T folhas e pesos w_j:

$$\Omega(f_t) = \gamma T + \frac{\lambda}{2} \sum_{j=1}^{T} w_j^2.$$

Na classificação binária, a margem $F(x_i)$ produz a probabilidade:

$$p_i = \frac{1}{1 + \exp[-F(x_i)]}, \qquad \ell(y_i, F_i) = -y_i \log p_i - (1 - y_i) \log(1 - p_i).$$

A expansão de Taylor de segunda ordem fornece:

$$\widetilde{\mathcal{L}}^{(t)} = \sum_{i=1}^{n} \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t), \qquad g_i = p_i - y_i, \quad h_i = p_i(1 - p_i).$$

Para a folha j, sejam $G_j = \sum_{i \in I_j} g_i$ e $H_j = \sum_{i \in I_j} h_i$. Sem penalidade L_1, o peso ótimo é:

$$w_j^\star = -\frac{G_j}{H_j + \lambda}.$$

Pesos por observação multiplicam os respectivos gradientes e hessianos. A parametrização de regularização, a função objetivo e a política para valores ausentes integram a identificação do modelo.

1.3.2 LightGBM

O LightGBM emprega estratégias de engenharia para reduzir custo de treino. Gradient-based One-Side Sampling preserva observações com gradientes elevados e amostra parte das demais, com correção de contribuição. Exclusive Feature Bundling combina variáveis predominantemente exclusivas para reduzir a dimensão processada. Os ganhos dependem da distribuição dos dados e da configuração (Ke et al., 2017).

XGBoost e LightGBM constituem alternativas experimentais. A escolha exige comparação sob a mesma amostragem, orçamento computacional e métrica de negócio. Superioridade universal não constitui premissa metodológica.

1.4 Linguagem de orquestração e motores computacionais

A arquitetura distribui responsabilidades por camada:

Camada	Responsabilidade
Oracle	Integridade transacional, filtros, junções, agregações e persistência
R e targets	Dependências, parametrização, coordenação e registro da execução
XGBoost ou outro estimador	Ajuste e avaliação do modelo
Bibliotecas compiladas	Operações numéricas, paralelismo e gerenciamento de memória
Formatos e contratos	Intercâmbio, preservação e validação

R, Python e Julia oferecem interfaces para motores compilados. O tempo total depende do algoritmo, das representações de dados, das cópias, dos kernels e do I/O. A linguagem de alto nível também pode introduzir custos; o desacoplamento reduz a dependência de uma implementação específica (Chen; Guestrin, 2016; Landau, *[S.d.]*).

A presença de GBDT em Oracle Machine Learning, IBM SPSS, interfaces C/C++, R, Python, Julia, Java ou Scala deve ser examinada por produto e versão. Intel oneDAL, NVIDIA RAPIDS e AMD ROCm representam ecossistemas de aceleração com cobertura própria. Disponibilidade de um algoritmo não implica APIs, parâmetros ou artefatos equivalentes (Oracle Corporation, *[S.d.]*, *[S.d.]*).

1.5 Expansibilidade e saídas

Tarefa	Formulação ou método	Saída
Classificação binária	Perda logística	Classe, margem e probabilidade
Regressão	Perda quadrática ou Huber	Valor contínuo
Classificação multiclasse	Softmax e entropia cruzada	Classe e vetor de probabilidades
Learning to Rank	LambdaRank ou LambdaMART	Score e ordenação por grupo
Modelagem hierárquica	Estrutura por níveis da carteira	Predições por entidade e agregação

Para resíduo $e = y - \hat{y}$, a perda de Huber combina comportamento quadrático e linear. Para K classes, Softmax normaliza os scores (Hastie; Tibshirani; Friedman, 2009):

$$L_\delta(e) = \begin{cases} \frac{1}{2}e^2, & |e| \le \delta, \\ \delta\left(|e| - \frac{\delta}{2}\right), & |e| > \delta, \end{cases} \qquad P(Y = k \mid x) = \frac{\exp F_k(x)}{\sum_{c=1}^{K} \exp F_c(x)}.$$

A matriz $P \in [0,1]^{N \times K}$ preserva as probabilidades completas por classe, com $\sum_k P_{ik} = 1$. Probabilidade produzida pelo algoritmo e probabilidade calibrada são conceitos distintos. A calibração exige ajuste e avaliação próprios. Modelos hierárquicos exigem explicitação das relações entre níveis e da consistência das agregações.

2 Governança e qualidade de dados

2.1 Dimensões e responsabilidades

DAMA-DMBOK organiza disciplinas, responsabilidades e práticas de gestão de dados. A segunda edição e sua revisão de 2024 constituem referências. As seis dimensões abaixo formam o conjunto operacional adotado no projeto; não representam uma taxonomia exclusiva ou imutável do framework (DAMA International, 2017, *[S.d.]*).

Dimensão	Definição operacional	Exemplo bancário
Completude	Presença dos valores necessários	Identificação do cliente preenchida
Validade	Conformidade com domínio e regra	Código de risco pertencente ao conjunto permitido
Acurácia	Correspondência com o fato representado	Saldo conciliado com a fonte contábil
Consistência	Compatibilidade entre representações	Totais mensais coerentes entre fontes
Tempestividade	Disponibilidade no prazo de uso	Competência carregada antes da predição
Unicidade	Ausência de duplicidade na granularidade definida	Uma observação por chave de negócio

Uma regra deve declarar população aplicável, unidade de teste, tratamento de ausentes, severidade e ação. Acurácia exige referência confiável; pertencer a um intervalo numérico comprova validade, sem comprovar acurácia.

2.2 Data Quality Score

Para M regras, sejam $N_j > 0$ as unidades aplicáveis, $n_{\text{pass},j}$ as aprovações e $n_{\text{fail},j}$ as reprovações. Com pesos $w_j \geq 0$ e $\sum_j w_j > 0$:

$$S_j = \frac{n_{\text{pass},j}}{N_j} = 1 - \frac{n_{\text{fail},j}}{N_j}, \qquad DQ_{\text{global}} = \frac{\sum_{j=1}^{M} w_j S_j}{\sum_{j=1}^{M} w_j}.$$

A igualdade pressupõe classificação de todas as unidades aplicáveis em aprovação ou reprovação. Regras sem população recebem estado **não aplicável**,

com política explícita de exclusão do denominador. Falha de execução da validação recebe estado próprio e impede declarar aprovação.

O score é uma medida institucional proposta, associada às dimensões de qualidade. Não constitui fórmula prescrita pelo DAMA-DMBOK. Uma regra crítica pode bloquear o processamento independentemente da média ponderada.

2.3 pointblank e gates

pointblank oferece agentes de validação, regras e resultados por etapa. A execução sobre tabelas remotas depende do suporte do backend e da tradução SQL. Regras suportadas podem ser avaliadas próximas aos dados, evitando extração integral (Iannone, *[S.d.]*).

Estado	Condição institucional	Ação
Aprovado	Regras críticas aprovadas e limites atendidos	Liberar etapa seguinte
Warning	Desvio tolerável segundo regra documentada	Registrar e sinalizar
Failure	Regra crítica ou limite impeditivo violado	Bloquear treinamento ou publicação
Não avaliado	Erro técnico ou ausência de evidência	Impedir aprovação automática

O registro da validação deve permitir reconstruir regra, versão, população, resultado e decisão aplicada.

No cenário inicial, os percentuais 99,2%, 98,4% e 100%, com pesos 3, 2 e 1, produzem $DQ_{\text{global}} \approx 99,07\%$. O valor agregado não substitui a avaliação individual das regras críticas.

3 Modelagem de dados e álgebra relacional

3.1 Transformações da Feature Store

Uma relação representa tuplas sob um esquema. A álgebra relacional permite descrever transformações independentemente da interface de programação. SQL acrescenta particularidades como duplicatas, nulidade e semântica de agregação (Codd, 1970; Oracle Corporation, *[S.d.]*).

Operador	Expressão	Aplicação
Seleção	$\sigma_{\varphi}(R)$	Elegibilidade e corte temporal
Projeção	$\pi_{A}(R)$	Escolha dos atributos necessários
Theta-Join	$R \bowtie_{\theta} S$	Associação por chaves e condições
Natural Join	$R \bowtie S$	Junção por atributos de mesmo nome
Agregação estendida	$\gamma_{G,F(A)}(R)$	Medidas por cliente e competência

A projeção seleciona atributos; a eliminação de colinearidade exige análise estatística adicional. A agregação estendida complementa os operadores clássicos. Junções devem declarar cardinalidade e preservar a granularidade esperada, evitando multiplicação indevida de valores monetários.

Cortes temporais e janelas devem usar limites explícitos. Intervalos semiabertos, como $[t_{\text{início}}, t_{\text{fim}})$, reduzem ambiguidade entre competências consecutivas.

3.2 Covariância e correlação

Para matriz $X \in \mathbb{R}^{n \times p}$, com linhas observacionais e média vetorial $\bar{x}$, a covariância amostral e a correlação de Pearson são:

$$S = \frac{1}{n-1}(X - \mathbf{1}\bar{x}^{\top})^{\top}(X - \mathbf{1}\bar{x}^{\top}), \qquad r_{ab} = \frac{\sum_i (x_{ia} - \bar{x}_a)(x_{ib} - \bar{x}_b)}{\sqrt{\sum_i (x_{ia} - \bar{x}_a)^2 \sum_i (x_{ib} - \bar{x}_b)^2}}.$$

Defina a população efetiva antes do cálculo: exclusão por par, por linha ou tratamento de ausentes produz resultados distintos. Variáveis constantes não possuem correlação de Pearson definida. No Oracle, `CORR` permite calcular correlação na origem dos dados; a avaliação de muitos pares exige controle do custo total (Hastie; Tibshirani; Friedman, 2009; Oracle Corporation, *[S.d.]*).

3.3 Entidades do projeto

As denominações abaixo substituem os nomes conceituais anteriores. O escopo resume exclusivamente as definições estabelecidas para o projeto.

Entidade	Escopo estabelecido
`ML_METHOD_LIBRARY`	Registro referencial dos métodos contemplados no processo e utilizados no pipeline
`ML_FEATURE_LINEAGE`	Origem das features, SQL de construção e decomposição das transformações
`ML_FEATURE_STORE`	Registro das features
`ML_MODEL_REGISTRY`	Registro do modelo
`ML_MODEL_ARTIFACT`	Armazenamento polimórfico dos artefatos do modelo
`ML_MODEL_TARGET`	Histórico da variável alvo utilizada no treinamento
`ML_PREDICTION_RESULT`	Resultados das predições

O desenho dimensional ou normalizado deve considerar fontes cadastrais, fatos transacionais, variáveis consolidadas e registros de governança. Particionamento e índices dependem das chaves e dos acessos definidos na modelagem física.

3.4 Modelo Entidade-Relacionamento



Figura 3.1: Espaço reservado para a imagem do Modelo Entidade-Relacionamento.

3.5 Dicionário de tabelas e campos

3.5.1 ML_METHOD_LIBRARY

3.5.2 ML_FEATURE_LINEAGE

3.5.3 ML_FEATURE_STORE

3.5.4 ML_MODEL_REGISTRY

3.5.5 ML_MODEL_ARTIFACT

3.5.6 ML_MODEL_TARGET

3.5.7 ML_PREDICTION_RESULT

4 Infraestrutura de alto desempenho

4.1 Hardware, sistema operacional e memória

O ambiente de referência informado compreende Oracle Linux, dois processadores Intel Xeon Platinum 8260, arquitetura Cascade Lake, e 1,4 TB de RAM DDR4 ECC Registered. Esses parâmetros descrevem o ambiente fornecido para o projeto; medições de capacidade disponível, topologia NUMA e configuração ativa devem acompanhar cada benchmark. O processador oferece instruções AVX-512 (Intel Corporation, *[S.d.]*).

A localidade NUMA influencia a latência e a largura de banda. Processos que alocam memória em um nó e executam em outro podem elevar o tráfego entre sockets. Mais threads não asseguram maior vazão quando memória ou I/O constituem o gargalo.

TuneD aplica perfis de desempenho. `throughput-performance` prioriza vazão em cargas de servidor; `latency-performance` prioriza resposta previsível com maior consumo de energia. Os perfis são alternativas de configuração, não camadas cumulativas. A escolha exige benchmark representativo e validação administrativa (Oracle Corporation, *[S.d.]*).

4.2 Intel oneAPI e concorrência

Componente	Papel	Critério de utilização
DPC++/C++ Compiler e Fortran Compiler	Compilação nativa	Compatibilidade, vetorização e resultado numérico
AVX-512	Processamento vetorial	Operações vetorizáveis e benefício medido
VNNI	Operações inteiras de inferência	Uso pelo kernel e representação suportada
oneMKL	BLAS e LAPACK	Operações de álgebra linear que utilizem essas interfaces
oneTBB	Paralelismo por tarefas	Integração efetiva com o componente
OpenMP	Paralelismo de memória compartilhada	Orçamento de threads e runtime coerente
mimalloc	Alocação de memória	Benefício observado na carga real

Compiladores, oneMKL e oneTBB atendem funções diferentes. Linkar uma biblioteca não converte automaticamente todos os algoritmos em usuários dessa biblioteca. O treino de árvores não deve ser caracterizado como operação BLAS apenas porque o R utiliza oneMKL (Intel Corporation, *[S.d.]*, *[S.d.]*, *[S.d.]*).

O mimalloc organiza a alocação por estruturas que favorecem concorrência e localidade. A substituição de `malloc` exige verificação de consumo, fragmentação, estabilidade e tempo; ganhos publicados não constituem medição do ambiente da instituição (Leijen; Zorn; Moura, 2019).

4.3 Compilação e controle numérico

Exemplo ilustrativo de Makevars para compilação local em processador com AVX-512. A configuração efetiva deve preservar as demais opções do ambiente R e as dependências do pacote (Intel Corporation, *[S.d.]*, *[S.d.]*):

```
CC = icx
CXX = icpx
CXX17 = icpx
FC = ifx
CFLAGS = -O3 -xCORE-AVX512
CXXFLAGS = -O3 -xCORE-AVX512
CXX17FLAGS = -O3 -xCORE-AVX512
FCFLAGS = -O3 -xCORE-AVX512
SHLIB_OPENMP_CFLAGS = -fiopenmp
SHLIB_OPENMP_CXXFLAGS = -fiopenmp
SHLIB_OPENMP_FFLAGS = -fiopenmp
```

Exemplo de redução vetorial em C++, com dependência explícita de aritmética de ponto flutuante:

```
#include <cstddef>

double DotProduct(const double* x, const double* y, std::size_t n){
  double result = 0.0;
  #pragma omp simd reduction(+:result)
  for(std::size_t i = 0; i < n; ++i){
    result += x[i] * y[i];
  }
  return result;
}
```

A reassociação de somas pode alterar os últimos dígitos. Flags que relaxam semântica numérica, como opções de *fast math*, exigem avaliação própria. Binários compilados para instruções específicas do processador possuem portabilidade inferior à do código-fonte e dos formatos de modelo.

O paralelismo externo de R/targets e o interno de XGBoost, BLAS ou SQL devem compartilhar um orçamento. Um limite inicial é $W \times T \leq C_{\text{disponíveis}}$, com W processos e T threads por processo; memória e I/O podem exigir concorrência menor.

4.4 Processamento na origem e Oracle In-Memory

Filtros, junções e agregações massivas devem, como princípio arquitetural, ser executados onde os dados residem quando o plano e a infraestrutura oferecem vantagem. A decisão reduz transferência e cópias; não estabelece superioridade universal de um motor (Oracle Corporation, *[S.d.]*).

Database In-Memory mantém representação colunar adicional às estruturas de linhas. Scans e agregações podem explorar processamento vetorial. `VECTOR GROUP BY` utiliza transformações específicas do otimizador e não é assegurado para toda consulta. Disponibilidade, capacidade configurada e licenciamento devem ser identificados (Oracle Corporation, *[S.d.]*).

Consultas parametrizadas favorecem controle e reutilização. Os benchmarks devem registrar cardinalidade, plano, concorrência, tempo decorrido e bytes transferidos.

5 Governança bancária e auditoria

5.1 Aplicabilidade regulatória

A documentação sustenta avaliação e auditoria por meio de evidências. Conformidade institucional exige verificação do escopo, dos controles implementados e da redação normativa aplicável. Os normativos não prescrevem nominalmente XGBoost, Oracle, OpenLineage, Dublin Core ou JSON Schema.

Referência	Objeto pertinente	Resposta arquitetural proposta
Resolução CMN nº 4.557/2017	Estruturas de gerenciamento de riscos e capital	Responsabilidades, documentação e controles proporcionais ao uso
Circular nº 3.978/2020	Política, procedimentos e controles de PLD/FTP	Rastreabilidade e evidências nos casos abrangidos
Resolução CMN nº 4.949/2021	Relacionamento com clientes e usuários	Regras documentadas e avaliação dos efeitos das decisões
LGPD, artigo 20	Revisão e informações sobre decisões automatizadas	Identificação do modelo, critérios e informação compreensível

As referências devem ser examinadas em sua redação consolidada. Modelos comerciais, operacionais e prudenciais possuem finalidades e obrigações diferentes. Um modelo de evasão não se converte em modelo de capital apenas por utilizar dados bancários (Brasil, 2018; Brasil. Banco Central do Brasil, 2020; Brasil. Conselho Monetário Nacional, 2017, 2021).

O artigo 20 da LGPD prevê solicitação de revisão de decisões exclusivamente automatizadas que afetem interesses e informações claras sobre critérios e procedimentos, observadas as condições legais. Uma explicação SHAP isolada não comprova atendimento integral a esse direito (Brasil, 2018).

5.2 BCBS 239 e dados de risco

BCBS 239 contém 14 princípios: 11 dirigidos aos bancos e três à supervisão. Sua aplicabilidade original concentra-se em instituições sistemicamente importantes. A arquitetura pode adotar os princípios como referência de engenharia sem

declarar enquadramento regulatório não verificado (Basel Committee on Banking Supervision, 2013).

Grupo	Princípios
Governança e infraestrutura	Governança; arquitetura de dados e infraestrutura de TI
Agregação de dados de risco	Acurácia e integridade; completude; tempestividade; adaptabilidade
Reporte	Acurácia; abrangência; clareza e utilidade; frequência; distribuição
Supervisão	Revisão; medidas corretivas e de supervisão; cooperação entre supervisores

Quando os dados alimentarem apuração prudencial, a cadeia de transformação até RWA deve preservar definições, reconciliações e responsabilidade. A menção a RWA delimita um caso de aplicação, sem atribuí-lo aos modelos comerciais do projeto.

5.3 Risco de modelos e evidências

Premissas, população de desenvolvimento, limitações, validação independente, aprovação e condições de recalibração compõem a governança proposta. A exigência específica deve ser vinculada ao normativo e ao uso aplicável. A trilha documental deve reconstruir dados, métodos, código, parâmetros, resultados e decisão de disponibilização.

A Resolução BCB nº 396/2024 trata de alterações no Regimento Interno do BCB. Não fundamenta exigência geral de catálogo de dados ou linhagem na Fábrica Preditiva. Sua referência é mantida apenas para registrar a correção de enquadramento do material de origem (Brasil. Banco Central do Brasil, 2024).

6 Padrões de metadados, linhagem e contratos

6.1 OpenLineage, Dublin Core e JSON Schema

Os padrões atuam sobre dimensões complementares do mesmo recurso:

Padrão	Pergunta respondida	Representação
OpenLineage	De onde veio e como foi produzido?	Datasets, Jobs, dependências e transformações
Dublin Core	O que representa e quem responde pelo recurso?	Termos semânticos e metadados descritivos
JSON Schema	Quais estruturas e valores são aceitos?	Contrato de validação

Em `ML_FEATURE_LINEAGE`, a SQL de construção registra a transformação declarada. Sua decomposição relaciona a feature produzida às fontes e colunas utilizadas. Dublin Core fornece um vocabulário de referência para a camada semântica; os conceitos bancários específicos pertencem à ontologia institucional (Dublin Core Metadata Initiative, *[S.d.]*; OpenLineage Project, *[S.d.]*; Wright et al., 2022).

6.2 OpenLineage e decomposição da SQL

Dataset identifica um conjunto de dados, como uma tabela. **Job** identifica o processo de transformação. **Run** identifica uma ocorrência de execução. O par `namespace` e `name` identifica Jobs e Datasets no ecossistema (OpenLineage Project, *[S.d.]*).

A linhagem declarada pode ser representada por `JobEvent`, sem execução associada. Eventos `RunEvent` representam execuções. A definição da família de features não deve receber um identificador fictício de execução.

A **SQL Job Facet** preserva a consulta. A **Column Lineage Dataset Facet** associa colunas de saída às colunas de entrada. Dependências **DIRECT** produzem o valor; dependências **INDIRECT** influenciam sua seleção ou organização, como filtros e agrupamentos (OpenLineage Project, *[S.d.]*, *[S.d.]*).

O registro da SQL não assegura decomposição automática correta. A extração de linhagem deve considerar o dialeto Oracle, aliases, CTEs, expressões, joins e SQL dinâmica. A decomposição resultante exige validação.

6.3 Vocabulário Dublin Core

DCMI Metadata Terms estende o conjunto básico Dublin Core. ISO 15836-1 padroniza os elementos centrais; os termos estendidos devem ser referidos pela especificação DCMI correspondente (Dublin Core Metadata Initiative, *[S.d.]*; International Organization for Standardization, 2017a).

Termos	Uso no projeto
`dcterms:identifier`	Identificador persistente da família ou recurso
`dcterms:title`; `dcterms:description`	Nome e significado
`dcterms:creator`; `dcterms:publisher`	Responsabilidade de criação e disponibilização
`dcterms:valid`; `dcterms:available`	Vigência e disponibilidade
`dcterms:source`; `dcterms:provenance`	Fonte e histórico de custódia ou derivação
`dcterms:conformsTo`	Padrão ou contrato ao qual o recurso declara aderência

Em representações JSON-LD, prefixos devem ser associados a URIs. Agentes, recursos e contratos devem receber identificadores quando a semântica da propriedade exigir recursos relacionados.

6.4 JSON Schema e constraints

JSON Schema Draft 2020-12 é a versão de referência dos contratos do projeto. É uma especificação do projeto JSON Schema, publicada na forma de drafts; não deve ser apresentada como norma ISO ou RFC finalizada (Wright et al., 2022; Wright; Andrews; Hutton, 2022).

Requisito	Palavra-chave
Campo obrigatório	`required`
Inteiro ou número	`type: integer` ou `type: number`
Domínio fechado	`enum`
Limites inclusivos	`minimum` e `maximum`
Limites exclusivos	`exclusiveMinimum` e `exclusiveMaximum`
Granularidade	`multipleOf`
Nulidade permitida	União de tipo com `null`
Campos adicionais controlados	`additionalProperties`

Um inteiro pode representar categoria ordinal sem expressar, sozinho, seu significado ou sua ordem de negócio. Essa semântica deve ser documentada. `required` controla presença; aceitar `null` é decisão independente.

JSON Schema valida documentos JSON. A aplicação das mesmas regras a colunas relacionais exige constraints SQL ou validação operacional correspondente. No Oracle 19c, `IS JSON` verifica a formação do documento, sem oferecer a validação

nativa por JSON Schema das versões posteriores (Oracle Corporation, *[S.d.]*, *[S.d.]*).

6.4.1 Exemplo hipotético para ML_FEATURE_LINEAGE

O exemplo associa uma feature de saldo à fonte hipotética `BDQ.SALDO_CONTA`. Inclui SQL, dependências do valor, agrupamento, termos Dublin Core e contrato decimal. O documento externo combina esses blocos segundo um perfil do projeto; o bloco `openLineage` utiliza `JobEvent`. Os nomes dos blocos do perfil não definem campos físicos da tabela.


```
{
  "FEATURE_FAMILY": "SALDOS",
  "metadata": {
    "@context": {
      "dcterms": "http://purl.org/dc/terms/"
    },
    "@id": "urn:models.factory:feature-family:saldos:v1",
    "dcterms:identifier": "SALDOS_V1",
    "dcterms:title": "Saldos consolidados por cliente",
    "dcterms:description": "Saldo total das contas por cliente e data de referência, em
    ↪  reais.",
    "dcterms:conformsTo": {
      "@id": "urn:models.factory:feature-contract:saldos:v1"
    }
  },
  "openLineage": {
    "eventTime": "2026-10-20T13:00:00Z",
    "producer": "https://models.factory.example/lineage/1.0.0",
    "schemaURL": "https://openlineage.io/spec/2-0-2/OpenLineage.json#/$defs/JobEvent",
    "job": {
      "namespace": "models.factory",
      "name": "FEATURE_FAMILY_SALDOS",
      "facets": {
        "sql": {
          "_producer": "https://models.factory.example/lineage/1.0.0",
          "_schemaURL": "https://openlineage.io/spec/facets/1-0-
          ↪  0/SQLJobFacet.json#/$defs/SQLJobFacet",
          "query": "SELECT SEQUENCIA_CLIENTE, DATA_REFERENCIA, CAST(SUM(VALOR_SALDO) AS
          ↪  NUMBER(18,2)) AS VALOR_SALDO_TOTAL FROM BDQ.SALDO_CONTA GROUP BY
          ↪  SEQUENCIA_CLIENTE, DATA_REFERENCIA"
        }
      }
    },
    "inputs": [
      {
        "namespace": "oracle://db.example.invalid:1521/BDQDBD",
        "name": "BDQ.SALDO_CONTA"
      }
    ],
    "outputs": [
      {
        "namespace": "oracle://db.example.invalid:1521/BDQDBD",
        "name": "BDQ.ML_FEATURE_STORE",
        "facets": {
          "columnLineage": {
            "_producer": "https://models.factory.example/lineage/1.0.0",
            "_schemaURL": "https://openlineage.io/spec/facets/1-2-
            ↪  0/ColumnLineageDatasetFacet.json#/$defs/ColumnLineageDatasetFacet",
            "fields": {
```

```
            "VALOR_SALDO_TOTAL": {
              "inputFields": [
                {
                  "namespace": "oracle://db.example.invalid:1521/BDQDBD",
                  "name": "BDQ.SALDO_CONTA",
                  "field": "VALOR_SALDO",
                  "transformations": [
                    {
                      "type": "DIRECT",
                      "subtype": "AGGREGATION",
                      "description": "Soma dos saldos, convertida para NUMBER(18,2).",
                      "masking": false
                    }
                  ]
                }
              ]
            }
          },
          "dataset": [
            {
              "namespace": "oracle://db.example.invalid:1521/BDQDBD",
              "name": "BDQ.SALDO_CONTA",
              "field": "SEQUENCIA_CLIENTE",
              "transformations": [
                {
                  "type": "INDIRECT",
                  "subtype": "GROUP_BY",
                  "description": "Define a granularidade por cliente e competência.",
                  "masking": false
                }
              ]
            },
            {
              "namespace": "oracle://db.example.invalid:1521/BDQDBD",
              "name": "BDQ.SALDO_CONTA",
              "field": "DATA_REFERENCIA",
              "transformations": [
                {
                  "type": "INDIRECT",
                  "subtype": "GROUP_BY",
                  "description": "Define a granularidade por cliente e competência.",
                  "masking": false
                }
              ]
            }
          ]
        }
      }
    }
  ]
},
"featureSchema": {
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "urn:models.factory:feature-contract:saldos:v1",
  "type": "object",
  "properties": {
    "VALOR_SALDO_TOTAL": {
      "description": "Saldo consolidado, em reais; null quando não houver saldo
      ↪  conhecido.",
      "type": [
        "number",
        "null"
      ],
```

```
        "multipleOf": 0.01,
        "exclusiveMinimum": -10000000000000000,
        "exclusiveMaximum": 10000000000000000
      }
    },
    "required": [
      "VALOR_SALDO_TOTAL"
    ],
    "additionalProperties": false
  }
}
```

A SQL consolida saldos por cliente e competência. `inputFields` descreve a contribuição de `VALOR_SALDO` ao resultado. A dependência de agrupamento fica documentada em `dataset`. A regra `multipleOf: 0.01` expressa granularidade em centavos; validadores devem tratar a aritmética decimal de forma compatível.

O perfil possui um JSON Schema separado em `schemas/feature-lineage.schema.json`. O contrato de valores das features é armazenado em `featureSchema`. Validação dos metadados, validação dos valores e validação das facets são etapas distintas.

6.5 Interoperabilidade financeira e FAIR

ISO 20022 fornece modelos e definições para mensagens financeiras. SWIFT participa desse ecossistema de mensageria. Sua utilização depende das fontes e interfaces do projeto; metadados de Machine Learning não se tornam mensagens ISO 20022 por compartilhar terminologia bancária (ISO 20022 Registration Authority, *[S.d.]*).

FAIR orienta recursos localizáveis, acessíveis, interoperáveis e reutilizáveis. Identificadores, contratos, proveniência e formatos documentados favorecem esses objetivos. Acessibilidade admite autenticação e autorização; não pressupõe abertura pública de dados bancários (Wilkinson et al., 2016).

7 Engenharia de software e reprodutibilidade

7.1 Qualidade de produto e ciclo de vida

ISO/IEC 25010:2011 descrevia oito características de qualidade. A edição 2023 organiza nove características, com mudanças de terminologia e escopo. A correspondência deve ser explícita para evitar combinação de edições (International Organization for Standardization, 2011, 2023).

Característica da edição 2023	Aplicação arquitetural
Functional suitability	Requisitos e saídas atendidos
Performance efficiency	Tempo, recursos e capacidade
Compatibility	Integração e intercâmbio
Interaction capability	Uso compreensível e informação adequada
Reliability	Continuidade e recuperação
Security	Acesso, integridade e proteção
Maintainability	Modularidade e rastreabilidade das alterações
Flexibility	Adaptação e portabilidade
Safety	Controle de consequências indesejáveis

A edição 2011 empregava usabilidade e portabilidade em sua organização de oito características. As denominações históricas permanecem contextualizadas.

ISO/IEC/IEEE 12207 estrutura processos do ciclo de vida de software. A edição 2017 foi substituída pela edição 2026. A documentação adota controle de edição e associa desenvolvimento, verificação, validação, manutenção e auditoria às evidências produzidas (International Organization for Standardization, 2017b, 2026).

ISO/IEC 22989:2022 fornece conceitos e terminologia de IA. ISO/IEC 23053:2022 fornece o framework conceitual dos sistemas que utilizam Machine Learning. A edição desta última é 2022, corrigindo a indicação 2023 do material inicial (International Organization for Standardization, 2022a, 2022b).

7.2 MLOps, DataOps e DAGs

MLOps articula desenvolvimento, disponibilização, acompanhamento e governança de modelos. DataOps coordena a produção e a qualidade dos dados que sustentam o processo. No projeto, os termos delimitam responsabilidades operacionais; a organização concreta será detalhada no capítulo do pipeline.

targets representa dependências em um DAG e identifica resultados desatualizados a partir de seus mecanismos de rastreamento. A execução incremental evita recomputação desnecessária. Dados externos ao mecanismo de rastreamento exigem identificação explícita de versão ou conteúdo (Landau, *[S.d.]*).

7.3 Reprodutibilidade, integridade e imutabilidade

Propriedade	Evidência necessária
Integridade de bytes	Hash do conteúdo e verificação posterior
Proveniência	Fontes, transformações, responsáveis e versões
Reprodutibilidade	Dados, código, configuração e ambiente recuperáveis
Determinismo numérico	Condições de execução e tolerâncias verificadas
Imutabilidade	Política de versões e impedimento de alteração indevida

SHA-256 identifica alterações de conteúdo com elevada resistência a colisões. Um checksum sem referência confiável não comprova autoria nem impede substituição maliciosa. A proteção do registro e o controle de acesso complementam a verificação criptográfica (National Institute of Standards and Technology, 2015).

Cada modelo deve ser associado às versões do código, dos métodos, dos dados e do ambiente. Quando framework e personalização estiverem em repositórios distintos, os dois commits devem ser preservados. Sementes aleatórias não asseguram, sozinhas, identidade bit a bit em execução paralela ou entre plataformas.

8 Persistência e precisão no Oracle Database 19c EE

8.1 Responsabilidades e organização física

O Oracle preserva dados relacionais, metadados e artefatos sob transações e controles de acesso. O desenho combina integridade, desempenho de consulta e recuperabilidade. Particionamento, compressão e paralelismo devem ser avaliados por objeto e padrão de acesso (Oracle Corporation, *[S.d.]*, *[S.d.]*).

A organização por competência e cliente favorece cargas e consultas mensais quando coincide com filtros efetivos. Índices, subpartições, clustering e limites de partição dependem do MER e do dicionário ainda reservados. A definição física das sete entidades não é antecipada aqui.

Views especializadas podem expor cada tipo de artefato e seus metadados, preservando uma representação física polimórfica em `ML_MODEL_ARTIFACT`. A interpretação do BLOB depende do formato registrado, da versão do contrato e do leitor correspondente.

8.2 Política de precisão decimal e não uso de FLOAT

Valores monetários, limites, percentuais e demais medidas decimais persistidas devem possuir domínio explícito. `NUMBER(p,s)` declara precisão total p e escala s. O projeto não utiliza `FLOAT` como escolha de DDL para esses valores (Oracle Corporation, *[S.d.]*).

Tipo Oracle	Representação	Consequência
`NUMBER(p,s)`	Decimal; precisão e escala declaradas	Contrato explícito de armazenamento
`FLOAT(p)`	Subtipo de NUMBER; precisão declarada em dígitos binários	Escala decimal não declarada
`BINARY_FLOAT`	IEEE 754 de 32 bits	Aproximações binárias e precisão simples
`BINARY_DOUBLE`	IEEE 754 de 64 bits	Aproximações binárias e precisão dupla

`FLOAT` do Oracle não deve ser confundido com os tipos `BINARY_FLOAT` e `BINARY_DOUBLE`. A escolha de `NUMBER(p,s)` favorece clareza do domínio decimal. O arredondamento ocorre ao reduzir a escala; excesso de precisão pode causar erro. Assim, a definição do tipo deve acompanhar a regra de negócio.

Com escala s, o passo decimal é $\Delta = 10^{-s}$. Para arredondamento ao mais próximo, sem overflow:

$$Q_s(x) = 10^{-s}\,\mathrm{round}(10^s x), \qquad |Q_s(x) - x| \le \frac{1}{2}\,10^{-s}.$$

A regra para empates e o momento do arredondamento devem ser definidos. Arredondar cada parcela e depois somar pode produzir resultado diferente de somar e arredondar o total.

A persistência decimal não torna toda a computação decimal exata. Matrizes R, kernels do XGBoost e operadores ONNX podem utilizar ponto flutuante binário. Converter o resultado para `NUMBER` não recupera dígitos já perdidos. O contrato deve registrar escalas, conversões e tolerâncias de comparação entre ambientes.

8.3 SecureFiles e políticas de armazenamento

SecureFiles oferece políticas distintas para cache, logging, compressão e deduplicação. Cada opção responde a uma decisão independente (Oracle Corporation, *[S.d.]*):

Opção	Efeito e critério
`NOCACHE`	Evita usar o buffer cache para o conteúdo LOB; adequado a leituras pouco frequentes
`CACHE READS`	Alternativa para conteúdo com leitura recorrente
`LOGGING`	Preserva logging segundo a política de recuperação
`NOLOGGING`	Reduz logging de conteúdo nas operações aplicáveis; exige estratégia de recuperação compatível
`NOCOMPRESS`	Evita compressão adicional do LOB
`COMPRESS`	Avaliar ganho, custo de CPU e licenciamento
`KEEP_DUPLICATES`	Desabilita deduplicação do LOB
`DEDUPLICATE`	Compartilha conteúdo idêntico no escopo suportado do segmento LOB

Para Parquet já comprimido, `NOCOMPRESS` é um candidato inicial. Artefatos repetidos podem justificar deduplicação. O benefício exige bytes idênticos e considera as fronteiras físicas dos segmentos e partições; não deve ser estimado como deduplicação global de todo o banco.

`NOLOGGING` não constitui recomendação geral para registros de governança ou artefatos insubstituíveis. `FORCE LOGGING`, operações realizadas e políticas de backup modificam o efeito prático.

Exemplo didático de armazenamento, sem definir campos das entidades do projeto:

```
CREATE TABLE EXEMPLO_ARTEFATO (
    ID_EXEMPLO NUMBER(10) NOT NULL,
    CONTEUDO BLOB,
    CONSTRAINT PK_EXEMPLO_ARTEFATO PRIMARY KEY (ID_EXEMPLO)
)
LOB (CONTEUDO) STORE AS SECUREFILE (
    NOCACHE
    LOGGING
    NOCOMPRESS
    KEEP_DUPLICATES
);
```

A variante `NOCACHE NOLOGGING NOCOMPRESS KEEP_DUPLICATES` pode ser avaliada para conteúdo reconstruível e sob política específica. Não substitui a avaliação de recuperabilidade.

8.4 Formatos e catálogo polimórfico

Formato	Finalidade	Portabilidade
Parquet com Zstandard	Matrizes e dados tabulares	Especificação aberta; leitor compatível
UBJSON nativo do XGBoost	Preservação do estimador final	Interfaces compatíveis do XGBoost
ONNX	Representação adicional de intercâmbio do modelo	Operadores, opset e runtime compatíveis
JSON e YAML	Metadados e configurações	Contrato de tipos e interpretação definido
RDS, qs2 e fst	Objetos ou tabelas de trabalho do ambiente R	Dependência do ecossistema e das versões

O catálogo deve identificar formato, versão, compressão e contrato de desserialização. Templates para R, Python, Julia e C++ somente devem ser declarados quando houver leitor efetivo. Um formato legível em R não adquire interoperabilidade por constar de um catálogo.

Parquet é um formato de arquivo; Iceberg é um formato de tabela que organiza metadados, snapshots e arquivos de dados. São camadas distintas (Apache Software Foundation, *[S.d.]*, *[S.d.]*).

8.5 Explicabilidade e quantização

SHAP representa a predição por contribuições aditivas, sob as definições do explicador:

$$f(x) = \phi_0 + \sum_{j=1}^{p} \phi_j(x).$$

LIME ajusta uma aproximação interpretável local, ponderada pela proximidade à observação explicada. Seus coeficientes dependem da aproximação e da vizinhança; não possuem automaticamente a mesma semântica das contribuições SHAP (Lundberg; Lee, 2017; Ribeiro; Singh; Guestrin, 2016).

A escala da explicação deve ser preservada: margem, log-odds, probabilidade ou outra saída. Quantização é transformação com perda e exige validação do efeito nas explicações.

Para $q = \text{round}(10^4 x)$ em Int16 com sinal:

$$-32768 \leq q \leq 32767, \qquad \tilde{x} = \frac{q}{10^4}, \qquad |\tilde{x} - x| \leq 5 \times 10^{-5}.$$

A faixa reconstruída é $[-3{,}2768;\ 3{,}2767]$. Valores fora da faixa não podem ser convertidos silenciosamente. O processo deve rejeitar a conversão ou selecionar representação de maior alcance. Para p contribuições quantizadas, um limite conservador do erro na soma é $p/(2 \cdot 10^4)$.

Para $N = 3{,}5$ milhões e $p = 200$, existem 700 milhões de valores:

Representação de referência	Bytes por valor	Volume sem compressão
Float64 em memória	8	5,6 GB
Int16 em memória	2	1,4 GB
INT32 físico antes de encodings Parquet	4	2,8 GB

Parquet representa o tipo lógico Int16 sobre o primitivo INT32; encodings e compressão determinam o tamanho efetivo. Portanto, 1,4 GB não é uma garantia de tamanho do arquivo. A meta inferior a 150 MB por execução depende da distribuição e exige medição. Zstandard reduz tamanho sem alterar os valores já quantizados; não reverte a perda da quantização (Apache Software Foundation, *[S.d.]*).

8.6 Consulta de arquivos externos

Uma external table acessa arquivos por diretórios do servidor ou mecanismos de Object Storage disponíveis no access driver. `LOCATION` não recebe um ponteiro para BLOB de uma linha relacional. Conteúdo em BLOB deve ser obtido pelo leitor da aplicação ou materializado por um processo específico antes de ser tratado como arquivo externo (Oracle Corporation, *[S.d.]*).

9 Preservação dupla do modelo final

9.1 Modelo nativo do XGBoost

O modelo final possui uma identidade de negócio e duas representações persistidas: nativa do XGBoost e ONNX. A representação ONNX é derivada do modelo final validado; não identifica um segundo treinamento.

Em R, `raw` é um vetor de bytes, não o nome de um formato universal. `xgb.save.raw()` pode produzir bytes do modelo em UBJSON. O formato deve ser explicitado para evitar dependência de defaults e do formato binário legado (XGBoost Developers, *[S.d.]*):

```
library(xgboost)

modelRaw <- xgb.save.raw(finalModel, raw_format = "ubj")
restoredModel <- xgb.load.raw(modelRaw)
```

`finalModel` representa o estimador final já produzido pelo pipeline. O exemplo documenta a serialização, sem executar treinamento ou gravação no banco.

A documentação do XGBoost distingue **model IO**, voltado ao modelo, e **memory snapshot**, voltado ao estado de execução. O projeto fornece compatibilidade de leitura de modelos anteriores nas versões posteriores, mas não assegura essa estabilidade aos snapshots. `saveRDS()` de um Booster não substitui o formato nativo de preservação (XGBoost Developers, *[S.d.]*).

A configuração completa do treinamento, callbacks, código de funções objetivo personalizadas e ambiente devem ser preservados separadamente quando não integrarem o arquivo do modelo.

9.2 ONNX como representação de intercâmbio

ONNX descreve um grafo com operações, entradas, saídas e tensores. A especificação separa versão da representação intermediária, domínios de operadores e versões de opset. A interpretação depende da implementação compatível (ONNX Project, *[S.d.]*; ONNX Runtime Project, *[S.d.]*).

Benefício	Consequência para a preservação
Formato aberto	Representação inspecionável e documentada
Interfaces em diferentes linguagens	Menor dependência da linguagem de treinamento
Runtimes e plataformas distintos	Mais opções futuras de utilização

Benefício	Consequência para a preservação
Contratos de entrada e saída	Intercâmbio com semântica verificável
Representação adicional ao formato nativo	Alternativa de leitura e utilização ao longo do tempo

A conversão de XGBoost é contemplada por conversores como onnxmltools, sujeita às versões e aos recursos do modelo. A documentação de conversão utiliza ferramentas do ecossistema Python. Não se pressupõe uma função nativa do pacote R XGBoost para exportar ONNX (ONNX Project, *[S.d.]*).

A etapa de conversão pode ser acionada pela orquestração em R com dependências isoladas e versionadas. A construção desta documentação não requer Python. O conversor deve aceitar a versão do modelo nativo e as características utilizadas no treinamento.

9.3 Contrato de preservação e equivalência

O conjunto de artefatos deve preservar:

- Identidade do modelo e relação entre representação nativa e ONNX.
- Versão do XGBoost, formato nativo, conversor, ONNX, IR, opset e runtime.
- Nomes, ordem, tipos, categorias e tratamento de ausentes das entradas.
- Transformações anteriores ao estimador e posteriores às saídas.
- Classes, ordem das probabilidades, margem, calibração e limiar de decisão.
- Checksums, resultado da conversão e evidência de equivalência.

Pré-processamento e calibração podem integrar o grafo ONNX ou permanecer em artefatos com contrato explícito. Exportar apenas as árvores não preserva automaticamente o processo preditivo completo.

Para predições a_i no ambiente nativo e b_i em ONNX, um critério de equivalência numérica é:

$$|a_i - b_i| \leq \varepsilon_{\text{abs}} + \varepsilon_{\text{rel}}|a_i|.$$

As tolerâncias devem ser justificadas para o domínio. A comparação deve abranger valores próximos aos thresholds das árvores e ao limiar decisório, ausentes, categorias, extremos e ordem das classes. Diferenças pequenas em probabilidade podem alterar a classe perto do ponto de corte.

A aprovação exige, além do erro numérico, comparação das decisões e das métricas relevantes. Compatibilidade de operadores e opsets não comprova equivalência estatística. A conversão só deve ser declarada aprovada após validação com o modelo efetivamente treinado.

10 Evolução tecnológica e marcadores de portabilidade

10.1 Continuidade dos padrões

O suporte documentado no Oracle posterior ao 19c reforça a escolha de JSON Schema, ONNX e XGBoost como componentes de um ecossistema corporativo. Trata-se de evidência de adoção e continuidade tecnológica. A aceitação de um arquivo específico exige verificação adicional.

Recurso	Base 19c EE do projeto	Evidência posterior e significado
JSON	BLOB, CLOB ou VARCHAR2 com recursos SQL/JSON disponíveis	Tipo JSON nativo em versões posteriores
JSON Schema	Contrato aplicado pela camada de validação	Validação declarativa documentada nas versões 23ai/26ai
XGBoost	Treinamento externo no desenho adotado	Matriz OML identifica suporte a partir de 21c nas interfaces indicadas
ONNX	BLOB preservado e interpretado por ferramentas externas	Integração documentada com Oracle Machine Learning e ONNX Runtime
VECTOR	Recurso não pertencente à base 19c aqui adotada	Tipo e AI Vector Search em versões posteriores
Lakehouse	Integração conforme driver e serviço disponíveis	Evolução de acesso a arquivos e tabelas abertas

A disponibilidade depende de versão, RU, plataforma, produto e licenciamento. A documentação online pode redirecionar páginas de versões antigas; a referência deve identificar a edição efetivamente consultada (Oracle Corporation, *[S.d.]*, *[S.d.]*, *[S.d.]*, *[S.d.]*).

10.2 JSON Schema declarativo

Nas versões com suporte, a condição `IS JSON VALIDATE` permite associar validação estrutural à declaração de integridade. Fragmento ilustrativo:

```
CHECK (
    DOCUMENTO_JSON IS JSON VALIDATE
    '{"type":"object","required":["VALOR"],
      "properties":{"VALOR":{"type":"number","exclusiveMinimum":0}}}'
)
```

Esse recurso não pertence ao núcleo de validação JSON Schema do 19c. O subconjunto de palavras-chave e referências aceito pelo Oracle deve ser comparado ao contrato Draft 2020-12 do projeto. Igualdade de nome do padrão não assegura cobertura integral da implementação (Oracle Corporation, *[S.d.]*).

10.3 ONNX e XGBoost no ecossistema Oracle

Oracle documenta integração de ONNX Runtime e suporte ao algoritmo XGBoost. Essas disponibilidades oferecem um marcador adicional de portabilidade e maturidade das escolhas. O formato nativo XGBoost, o algoritmo disponível em OML e o suporte ONNX são recursos distintos (Oracle Corporation, *[S.d.]*, *[S.d.]*).

`DBMS_VECTOR.LOAD_ONNX_MODEL` é citado apenas como API documentada desse ecossistema. Operadores, metadados e tipos de modelo aceitos dependem da versão. Sua existência não comprova importação automática de qualquer XGBoost convertido (Oracle Corporation, *[S.d.]*).

O objetivo permanece a preservação de modelos altamente portáveis. Não se propõe migração do treinamento ou da inferência para o Oracle. A arquitetura continua utilizando o banco como plataforma de dados, integridade, governança e persistência.

10.4 Lakehouse e vetores

Iceberg organiza arquivos e metadados de tabelas, com evolução de esquema e particionamento. Parquet e ORC são formatos de arquivo; Delta Lake é formato de tabela. O suporte Oracle a cada combinação deve ser identificado por driver, versão e serviço (Apache Software Foundation, *[S.d.]*, *[S.d.]*; Oracle Corporation, *[S.d.]*).

`ORACLE_BIGDATA` e `DBMS_CLOUD` possuem contextos próprios. Predicate pushdown e column pruning dependem do acesso efetivo e do plano. Não se presume que qualquer instalação 19c EE acesse nativamente todos os formatos.

`VECTOR` e AI Vector Search atendem representação e busca vetorial. São recursos de evolução com finalidade distinta da preservação dupla do modelo final (Oracle Corporation, *[S.d.]*).

11 Matriz de aderência e evolução arquitetural

11.1 Requisitos, decisões e evidências

A matriz registra relações de engenharia propostas. O estado implementado depende de evidência do ambiente da instituição.

Requisito ou referência	Decisão arquitetural	Evidência esperada
Governança de riscos e capital	Responsabilidades e validação conforme aplicabilidade	Aprovação, revisão e limitações
Relacionamento com clientes e LGPD	Critérios e resultados rastreáveis	Identidade do modelo e informação compreensível
BCBS 239, quando aplicável	Integridade, conciliação e rastreabilidade	Fonte, transformação e reconciliação
DAMA-DMBOK	Regras por dimensão e responsáveis	Resultados e decisão dos gates
OpenLineage	Origem e decomposição das transformações	SQL e dependências por coluna
Dublin Core	Vocabulário semântico	Metadados identificados e versionados
JSON Schema	Contratos verificáveis	Esquemas, versões e resultados de validação
ISO/IEC 25010	Qualidade de produto	Requisitos e critérios mensuráveis
ISO/IEC/IEEE 12207	Controle do ciclo de vida	Revisões, validações e manutenção
Reprodutibilidade	Código, dados e configuração preservados	Commits, versões e identificadores
Integridade	SHA-256 e proteção do registro	Verificação dos artefatos
Precisão decimal	NUMBER(p,s) nos domínios definidos	DDL, escalas e testes de conversão
Preservação do modelo	XGBoost nativo e ONNX	Artefatos e teste de equivalência
HPC	Concorrência e compilação controladas	Benchmark e configuração
Persistência	Políticas de LOB por conteúdo	Tamanho, leitura e recuperação

11.2 Roadmap e estados

Etapa	Entrega verificável
Consolidar modelagem	Imagem ER, dicionário e contratos dos campos
Consolidar linhagem	Perfil validado, SQL e decomposição conferida
Medir infraestrutura	Baseline de tempo, memória, I/O e concorrência
Formalizar artefatos	Formatos, identificação, checksums e leitura
Validar portabilidade	Conversão ONNX e equivalência com modelo final
Documentar funções	Inclusão de arquivos Rd em docs/functions
Avaliar evolução	Matriz de compatibilidade por versão e produto

As decisões devem receber estado **definido**, **implementado**, **validado** ou **dependente de verificação**. Um recurso descrito na documentação oficial pode estar definido como opção futura sem estar disponível ou habilitado no ambiente atual.

12 Modelagem

13 Predictive Factory Pipeline

14 Referência de funções

Referências

APACHE SOFTWARE FOUNDATION. **Apache Parquet: File Format, Types, Logical Types and Compression**. *[S.l.: S.n.]*.

APACHE SOFTWARE FOUNDATION. **Apache Iceberg Table Specification**. *[S.l.: S.n.]*.

BASEL COMMITTEE ON BANKING SUPERVISION. **Principles for Effective Risk Data Aggregation and Risk Reporting**. *[S.l.]*: Bank for International Settlements, 2013. Disponível em: <https://www.bis.org/publ/bcbs239.htm>.

BRASIL. **Lei nº 13.709, de 14 de agosto de 2018: Lei Geral de Proteção de Dados Pessoais**., 2018. Disponível em: <https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709compilado.htm>. Acesso em: 20 out. 2026

BRASIL. BANCO CENTRAL DO BRASIL. **Circular nº 3.978, de 23 de janeiro de 2020**., 2020. Disponível em: <https://www.bcb.gov.br/estabilidadefinanceira/exibenormativo?numero=3978&tipo=Circular>. Acesso em: 20 out. 2026

BRASIL. BANCO CENTRAL DO BRASIL. **Resolução BCB nº 396, de 27 de junho de 2024: Alterações no Regimento Interno do Banco Central do Brasil**., 2024. Disponível em: <https://www.bcb.gov.br/estabilidadefinanceira/exibenormativo?numero=396&tipo=Resolu%C3%A7%C3%A3o+BCB>. Acesso em: 20 out. 2026

BRASIL. CONSELHO MONETÁRIO NACIONAL. **Resolução CMN nº 4.557, de 23 de fevereiro de 2017**., 2017. Disponível em: <https://www.bcb.gov.br/estabilidadefinanceira/exibenormativo?numero=4557&tipo=Resolu%C3%A7%C3%A3o>. Acesso em: 20 out. 2026

BRASIL. CONSELHO MONETÁRIO NACIONAL. **Resolução CMN nº 4.949, de 30 de setembro de 2021**., 2021. Disponível em: <https://www.bcb.gov.br/estabilidadefinanceira/exibenormativo?numero=4949&tipo=Resolu%C3%A7%C3%A3o+CMN>. Acesso em: 20 out. 2026

BRITO, João B. G. *et al.* A Framework to Improve Churn Prediction Performance in Retail Banking. **Financial Innovation**, v. 10, n. 17, 2024.

BRITO, João B. G. de *et al.* **Predicting and Explaining Customer Data Sharing in the Open Banking**. arXiv, 2025. Disponível em: <https://arxiv.org/abs/2507.01987>

CHEN, Tianqi; GUESTRIN, Carlos. XGBoost: A Scalable Tree Boosting System. *In*: ACM, 2016. Disponível em: <https://arxiv.org/abs/1603.02754>

CODD, Edgar F. A Relational Model of Data for Large Shared Data Banks. **Communications of the ACM**, v. 13, n. 6, p. 377–387, 1970.

DAMA INTERNATIONAL. **DAMA-DMBOK: Data Management Body of Knowledge**. 2. ed. Basking Ridge: Technics Publications, 2017.

DAMA INTERNATIONAL. **DAMA-DMBOK2 Revised Edition: FAQs**. *[S.l.: S.n.]*.

DUBLIN CORE METADATA INITIATIVE. **DCMI Metadata Terms**. *[S.l.: S.n.]*.

HASTIE, Trevor; TIBSHIRANI, Robert; FRIEDMAN, Jerome. **The Elements of Statistical Learning: Data Mining, Inference, and Prediction**. 2. ed. New York: Springer, 2009.

IANNONE, Richard. **pointblank: Data Validation and Organization of Metadata for Local and Remote Tables**. *[S.l.: S.n.]*.

INTEL CORPORATION. **Intel Xeon Platinum 8260 Processor: Specifications**. *[S.l.: S.n.]*.

INTEL CORPORATION. **Intel oneAPI DPC++/C++ Compiler Developer Guide and Reference**. *[S.l.: S.n.]*.

INTEL CORPORATION. **Compiling and Running an OpenMP Application**. *[S.l.: S.n.]*.

INTEL CORPORATION. **Intel oneAPI Math Kernel Library Developer Reference**. *[S.l.: S.n.]*.

INTEL CORPORATION. **Intel oneAPI Threading Building Blocks Developer Guide**. *[S.l.: S.n.]*.

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010:2011: Systems and Software Quality Requirements and Evaluation: System and Software Quality Models**., 2011. Disponível em: <https://www.iso.org/standard/35733.html>

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO 15836-1:2017: Information and Documentation: The Dublin Core Metadata Element Set: Part 1: Core Elements**., a2017. Disponível em: <https://www.iso.org/standard/71339.html>

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC/IEEE 12207:2017: Systems and Software Engineering: Software Life Cycle Processes**., b2017. Disponível em: <https://www.iso.org/standard/63712.html>

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 22989:2022: Information Technology: Artificial Intelligence: Artificial Intelligence Concepts and Terminology**., a2022. Disponível em: <https://www.iso.org/standard/74296.html>

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 23053:2022: Framework for Artificial Intelligence Systems Using Machine Learning**., b2022. Disponível em: <https://www.iso.org/standard/74438.html>

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010:2023: Systems and Software Engineering: SQuaRE: Product Quality Model**., 2023. Disponível em: <https://www.iso.org/standard/78176.html>

INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC/IEEE 12207:2026: Systems and Software Engineering: Software Life Cycle Processes**., 2026. Disponível em: <https://www.iso.org/standard/90219.html>

ISO 20022 REGISTRATION AUTHORITY. **ISO 20022: Universal Financial Industry Message Scheme**. *[S.l.: S.n.]*.

KE, Guolin *et al.* LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *In*: 2017. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html>

LANDAU, Will. **The targets R Package User Manual**. *[S.l.: S.n.]*.

LEIJEN, Daan; ZORN, Benjamin; MOURA, Leonardo de. **Mimalloc: Free List Sharding in Action**. *[S.l.]*: Microsoft Research, 2019. Disponível em: <https://www.microsoft.com/en-us/research/publication/mimalloc-free-list-sharding-in-action/>.

LUNDBERG, Scott M.; LEE, Su-In. A Unified Approach to Interpreting Model Predictions. *In*: 2017. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html>

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. **Secure Hash Standard**. *[S.l.: S.n.]*.

ONNX PROJECT. **ONNX: Open Neural Network Exchange: Intermediate Representation**. *[S.l.: S.n.]*.

ONNX PROJECT. **Converters**. *[S.l.: S.n.]*.

ONNX RUNTIME PROJECT. **ONNX Runtime: Compatibility**. *[S.l.: S.n.]*.

OPENLINEAGE PROJECT. **OpenLineage: Object Model**. *[S.l.: S.n.]*.

OPENLINEAGE PROJECT. **Column Level Lineage Dataset Facet**. *[S.l.: S.n.]*.

OPENLINEAGE PROJECT. **SQL Job Facet**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle Linux: Reviewing TuneD Profiles**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle Database Administrator's Guide, 19c**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle Database Performance Tuning Guide, 19c**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle Database In-Memory Guide, 19c: Optimizing Aggregation**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle Database SQL Language Reference, 19c: Data Types**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle Database SecureFiles and Large Objects Developer's Guide, 19c: Using Oracle LOB Storage**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle Database Utilities, 19c: External Tables Concepts**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle Database JSON Developer's Guide, 19c**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle AI Database JSON Developer's Guide: JSON Schema**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle Machine Learning for SQL: About ONNX**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle Machine Learning for SQL: About XGBoost**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle Machine Learning Products and Feature Matrix**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle AI Database PL/SQL Packages and Types Reference: DBMS_VECTOR**. *[S.l.: S.n.]*.

ORACLE CORPORATION. **Oracle Database: ORACLE_BIGDATA Access Driver**. *[S.l.: S.n.]*.

RIBEIRO, Marco Tulio; SINGH, Sameer; GUESTRIN, Carlos. Why Should I Trust You? Explaining the Predictions of Any Classifier. *In*: ACM, 2016. Disponível em: <https://arxiv.org/abs/1602.04938>

VAPNIK, Vladimir N. **Statistical Learning Theory**. New York: Wiley, 1998.

WILKINSON, Mark D. *et al*. The FAIR Guiding Principles for Scientific Data Management and Stewardship. **Scientific Data**, v. 3, p. 160018, 2016.

WRIGHT, Austin *et al*. **JSON Schema: A Media Type for Describing JSON Documents**., 2022. Disponível em: <https://json-schema.org/draft/2020-12/json-schema-core>. Acesso em: 20 out. 2026

WRIGHT, Austin; ANDREWS, Henry; HUTTON, Ben. **JSON Schema Validation:**

A Vocabulary for Structural Validation of JSON., 2022. Disponível em: <https://json-schema.org/draft/2020-12/json-schema-validation>. Acesso em: 20 out. 2026

XGBOOST DEVELOPERS. **Introduction to Model IO**. *[S.l.: S.n.]*.

XGBOOST DEVELOPERS. **Save XGBoost Model to R's Raw Vector: xgb.save.raw**. *[S.l.: S.n.]*.

ZOU, Hui; HASTIE, Trevor. Regularization and Variable Selection via the Elastic Net. **Journal of the Royal Statistical Society: Series B**, v. 67, n. 2, p. 301–320, 2005.